

GUIA DE EXECUÇÃO — Máquina de Vendas do Zero ao Deploy

Passo a passo completo: do clone do repositório até a máquina rodando em produção. Tempo estimado total: 2-4 horas (primeira vez), 30 minutos (a partir da segunda).

ÍNDICE

1. Pré-requisitos
 2. Clone e Configuração do Ambiente
 3. Criar uma Obra (ou usar existente)
 4. Gerar a Máquina de Vendas
 5. Configurar o Frontend
 6. Configurar o Backend
 7. Configurar o Banco de Dados
 8. Configurar as Automações
 9. Configurar Integrações Externas
 10. Testes Locais
 11. Deploy em Produção
 12. Pós-Deploy: Operação 24/7
 13. Monitoramento e Manutenção
 14. Escalando a Máquina
-

1. PRÉ-REQUISITOS

1.1 Software Obrigatório

Software	Versão	Como instalar	Verificar
Python	3.10+	python.org	<code>python --version</code>
Node.js	18+	nodejs.org	<code>node --version</code>
npm	9+	vem com Node	<code>npm --version</code>
Git	2.30+	git-scm.com	<code>git --version</code>
Pandoc	2.17+	pandoc.org	<code>pandoc --version</code>
Typst	0.8+	<code>cargo install typst</code> ou download	<code>typst --version</code>

1.2 Software Opcional (para deploy)

Software	Para que	Como instalar
Docker	Deploy containerizado	docker.com
Docker Compose	Orquestração	vem com Docker Desktop
Vercel CLI	Deploy frontend	<code>npm i -g vercel</code>
Railway CLI	Deploy backend	<code>npm i -g @railway/cli</code>

1.3 Contas e API Keys

Serviço	Para que	Onde obter	Custo
LLM (obrigatório)	Geração de conteúdo	MiMoCode / Claude Code / Gemini CLI	Varia
E-mail SMTP	Envio de e-mails	Gmail (App Password) ou SendGrid	Grátis~\$15/mês
Stripe	Pagamentos	dashboard.stripe.com	2.9% + R\$0.30/transação
Instagram Graph API	Busca de leads	developers.facebook.com	Grátis
ElevenLabs	Áudio narrado (opcional)	elevenlabs.io	\$5/mês (starter)
DALL-E 3	Imagens (opcional)	platform.openai.com	\$0.04/imagem
Vercel	Deploy frontend	vercel.com	Grátis (Hobby)
Railway	Deploy backend	railway.app	\$5/mês

1.4 Hardware

Configuração	Mínimo	Recomendado
RAM	4 GB	8 GB
Disco	2 GB livre	10 GB livre
CPU	2 cores	4 cores
OS	Windows 10+, macOS 12+, Ubuntu 20+	Qualquer

2. CLONE E CONFIGURAÇÃO DO AMBIENTE

2.1 Clonar o repositório

```
git clone https://github.com/Heverton-web/proj_fabrica-de-livros.git
cd proj_fabrica-de-livros/vendas
```

2.2 Instalar dependências Python

```
pip install -r requirements.txt
```

Saída esperada:

```
Successfully installed Pillow-10.x.x playwright-1.x.x
```

2.3 Instalar dependências do Playwright (para capas/ilustrações)

```
playwright install chromium
```

2.4 Verificar configuração

```
python scripts/descobrir_modelos.py
```

Saída esperada:

```
=====
DESCOBERTA DE LLMs – Diagnóstico do Harness
```

```
=====
🔍 Harness detectado: mimocode, claude_code
🤖 Runtime Orca/MiMoCode detectado
Modelo da sessão: mimo-v2.5-pro

📦 Modelos disponíveis:
ORCA_RUNTIME (4 modelos):
  lite: mimo-v2.5-lite
  standard: mimo-v2.5, mimo-v2.5-standard
  pro: mimo-v2.5-pro
=====
```

```
=====
RESUMO: 4 modelos | 21/24 tarefas roteáveis
=====
```

2.5 Criar arquivo .env

```
cp .env.example .env
```

Editar .env com suas credenciais:

```
# OBRIGATÓRIO para e-mails
SMTP_HOST=smtp.gmail.com
SMTP_PORT=587
SMTP_USER=seu@email.com
SMTP_PASS=sua-app-password

# OBRIGATÓRIO para pagamentos
STRIPE_SECRET_KEY=sk_test_...
STRIPE_WEBHOOK_SECRET=whsec_...

# OBRIGATÓRIO para busca de leads
INSTAGRAM_ACCESS_TOKEN=EAx...

# OPCIONAL para áudio narrado
ELEVENLABS_API_KEY=...

# OPCIONAL para geração de imagens
DALL_E_API_KEY=...

# FRONTEND
NEXT_PUBLIC_API_URL=http://localhost:8000
NEXT_PUBLIC_STRIPE_KEY=pk_test_...
```

3. CRIAR UMA OBRA (ou usar existente)

3.1 Verificar obras existentes

```
ls output/livros/
ls output/tccs/
ls output/ebooks/
```

Se já tem uma obra criada, pule para o passo 4.

3.2 Criar nova obra

```
# Via comando slash (no MiMoCode/Claude Code)
/criar-livro Inteligência Artificial para Empreendedores
```

```
# Ou via script direto
python scripts/parametros_obra.py "Inteligência Artificial para Empreendedores"
```

3.3 Acompanhar criação

A obra será criada em output/livros/inteligencia-artificial-empreendedores/:

```
output/livros/inteligencia-artificial-empreendedores/
├── capitulos/           # Capítulos em Markdown
├── pesquisa/           # Dossiê técnico
├── artes/              # Ilustrações
├── config_obra.json    # Configurações
├── sumario_macro.json  # Sumário
├── inteligencia-artificial-empreendedores.pdf
└── inteligencia-artificial-empreendedores.epub
```

Tempo: 30-60 minutos (autônomo).

3.4 Validar obra

```
python scripts/auditar-obra.py inteligencia-artificial-empreendedores --estrito
```

Saída esperada:

✅ Obra CONFORME – todos os 14 requisitos atendidos

4. GERAR A MÁQUINA DE VENDAS

4.1 Executar comando

```
# Via comando slash
/criar-maquina inteligencia-artificial-empreendedores
```

```
# Ou via script direto
python scripts/criar-maquina-vendas.py inteligencia-artificial-empreendedores --tipo
completo
```

4.2 Acompanhar geração

```
=====
CRIANDO MÁQUINA DE VENDAS: Inteligência Artificial Para Empreendedores
Tipo: completo
Destino: marketing/maquinas/inteligencia-artificial-empreendedores
=====

[1/6] Copiando estrutura de templates...
[2/6] Gerando manifesto...
[3/6] Copiando conteúdo da obra...
[4/6] Inicializando banco de dados...
    ✅ Banco criado: marketing/maquinas/.../database/maquina.db
[5/6] Gerando .mcp.json...
[6/6] Gerando resumo...

=====
```

✅ MÁQUINA CRIADA COM SUCESSO!
📁 marketing/maquinas/inteligencia-artificial-empreendedores
📄 83 arquivos gerados

PRÓXIMOS PASSOS:

1. `cd marketing/maquinas/inteligencia-artificial-empreendedores`
2. Revisar `config/*.json`
3. Configurar `.env`
4. `cd frontend && npm install && npm run dev`
5. `cd backend && pip install -r requirements.txt && uvicorn app.main:app`
6. Deploy: `bash scripts/deploy.sh`

=====

4.3 Verificar estrutura gerada

```
cd marketing/maquinas/inteligencia-artificial-empreendedores
dir # Windows
ls # Mac/Linux

.env.example
AGENTS.md
CLAUDE.md
README.md
SPEC.md
config/
database/
docker-compose.yml
frontend/
backend/
manifesto.json
scripts/
templates/
vercel.json
```

5. CONFIGURAR O FRONTEND

5.1 Entrar no diretório

```
cd frontend
```

5.2 Instalar dependências

```
npm install
```

Saída esperada:

added 342 packages in 28s

5.3 Configurar variáveis de ambiente

```
cp .env.example .env.local
```

Editar `.env.local`:

```
NEXT_PUBLIC_API_URL=http://localhost:8000
NEXT_PUBLIC_STRIPE_KEY=pk_test_sua_chave
```

5.4 Personalizar conteúdo

Editar `app/page.tsx` — substituir placeholders:

```
// Trocar {{TITULO}} pelo título real
<h1>Inteligência Artificial para Empreendedores</h1>

// Trocar {{PRECO}} pelo preço real
<span>R$ 97</span>

// Trocar {{DESCRICA0}} pela descrição real
<p>Domine IA aplicada a negócios e saia na frente da concorrência</p>
```

5.5 Iniciar servidor de desenvolvimento

```
npm run dev
```

Saída esperada:

```
▲ Next.js 14.x.x
- Local: http://localhost:3000
```

✓ Ready in 2.3s

5.6 Testar páginas

Abrir no navegador:

URL	Página	O que verificar
http://localhost:3000	Venda	Hero, preço, CTA visíveis
http://localhost:3000/captura	Captura	Formulário funcional
http://localhost:3000/obrigado	Agradecimento	Mensagem visível
http://localhost:3000/checkout	Checkout	Preço correto
http://localhost:3000/admin	Dashboard	Cards de métricas
http://localhost:3000/api/health	Health	{"status": "ok"}

6. CONFIGURAR O BACKEND

6.1 Abrir novo terminal

```
cd marketing/maquinas/inteligencia-artificial-empresendedores/backend
```

6.2 Criar ambiente virtual

```
python -m venv venv
```

```
# Windows
venv\Scripts\activate
```

```
# Mac/Linux
source venv/bin/activate
```

6.3 Instalar dependências

```
pip install -r requirements.txt
```

Saída esperada:

```
Successfully installed fastapi-0.x.x uvicorn-0.x.x pydantic-settings-x.x.x httpx-0.x.x
```

6.4 Configurar variáveis de ambiente

```
# Windows
set DATABASE_URL=sqlite:///../database/maquina.db
set SMTP_HOST=smtp.gmail.com
set SMTP_PORT=587
set SMTP_USER=seu@email.com
set SMTP_PASS=sua-app-password
set STRIPE_SECRET_KEY=sk_test_...
set STRIPE_WEBHOOK_SECRET=whsec_...

# Mac/Linux
export DATABASE_URL=sqlite:///../database/maquina.db
export SMTP_HOST=smtp.gmail.com
export SMTP_PORT=587
export SMTP_USER=seu@email.com
export SMTP_PASS=sua-app-password
export STRIPE_SECRET_KEY=sk_test_...
export STRIPE_WEBHOOK_SECRET=whsec_...
```

6.5 Iniciar servidor

```
uvicorn app.main:app --reload --host 0.0.0.0 --port 8000
```

Saída esperada:

```
INFO:      Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
INFO:      Started reloader process
INFO:      Started server process
INFO:      Waiting for application startup.
INFO:      Application startup complete.
```

6.6 Testar API

```
# Health check
curl http://localhost:8000/health

# Listar leads
curl http://localhost:8000/api/leads

# Criar lead
curl -X POST http://localhost:8000/api/leads \
  -H "Content-Type: application/json" \
  -d '{"nome": "João Silva", "email": "joao@teste.com", "fonte": "instagram"}'

# Métricas do funil
curl http://localhost:8000/api/funil/metricas
```

6.7 Acessar documentação automática

Swagger UI: <http://localhost:8000/docs>

ReDoc: <http://localhost:8000/redoc>

7. CONFIGURAR O BANCO DE DADOS

7.1 Verificar schema

```
sqlite3 database/maquina.db ".tables"
```

Saída esperada:

campanhas
emails_enviados
interacoes
leads
metricas_diarias
vendas

7.2 Verificar dados de exemplo

```
sqlite3 database/maquina.db "SELECT * FROM leads LIMIT 5;"
```

7.3 Limpar dados de exemplo (opcional)

```
sqlite3 database/maquina.db "  
DELETE FROM interacoes;  
DELETE FROM vendas;  
DELETE FROM emails_enviados;  
DELETE FROM metricas_diarias;  
DELETE FROM leads;  
DELETE FROM campanhas;  
"
```

7.4 Backup automático

O script funnel_monitor.py faz backups diários em database/backups/.

Backup manual:

```
cp database/maquina.db database/backups/maquina_$(date +%Y%m%d).db
```

8. CONFIGURAR AS AUTOMAÇÕES

8.1 Lead Hunter (busca leads no Instagram)

Editar config/personas.json:

```
{  
  "personas": [  
    {  
      "nome": "Empreendedor Tech",  
      "hashtags": ["#empreendedorismo", "#ia", "#tecnologia", "#startup"],  
      "faixa_seguidores": {"min": 500, "max": 50000},  
      "localizacao": "Brasil"  
    }  
  ]  
}
```

Editar config/canais.json:

```
{  
  "instagram": {  
    "limite_diario_dm": 20,  
    "intervalo_entre_dm_minutos": 5,  
    "horario_envio": {"inicio": "08:00", "fim": "22:00"}  
  }  
}
```


8.2 Email Sender (dispara e-mails)

Editar config/email.json:

```
{
  "smtp": {
    "host": "smtp.gmail.com",
    "port": 587,
    "user_env": "SMTP_USER",
    "pass_env": "SMTP_PASS"
  },
  "remetente": {
    "nome": "Fábrica de Livros",
    "email": "contato@seudominio.com"
  },
  "limites": {
    "max_emails_dia": 100,
    "intervalo_entre_emails_segundos": 30
  }
}
```

8.3 Funnel Monitor (monitoramento 24/7)

Editar config/funis.json para ajustar thresholds:

```
{
  "thresholds": {
    "taxa_captura_min": 0.02,
    "taxa_email_open_min": 0.20,
    "taxa_venda_min": 0.01,
    "alerta_queda_percentual": 20
  }
}
```

8.4 Auto Correct (correção automática)

Editar config/subagentes.json:

```
{
  "auto_correct": {
    "ativo": true,
    "espera_horas": 48,
    "min_amostra": 50
  }
}
```

9. CONFIGURAR INTEGRAÇÕES EXTERNAS

9.1 Stripe (pagamentos)

1. Criar conta em dashboard.stripe.com
2. Obter API keys (Settings → API Keys)
3. Configurar webhook:
 - URL: <https://seu-dominio.com/api/webhook>
 - Events: checkout.session.completed, payment_intent.succeeded

4. Adicionar em .env:

```
STRIPE_SECRET_KEY=sk_test_...  
STRIPE_WEBHOOK_SECRET=whsec_...
```

9.2 Instagram Graph API (leads)

1. Criar app em developers.facebook.com
2. Configurar Instagram Graph API
3. Obter access token de longa duração
4. Adicionar em .env:

```
INSTAGRAM_ACCESS_TOKEN=EAAx...
```

9.3 SendGrid (e-mails alternativo)

1. Criar conta em sendgrid.com
2. Obter API key (Settings → API Keys)
3. Verificar domínio (Settings → Sender Authentication)
4. Adicionar em .env:

```
SENDGRID_API_KEY=SG.xxxx
```

9.4 ElevenLabs (áudio narrado)

1. Criar conta em elevenlabs.io
2. Obter API key (Profile → API Key)
3. Adicionar em .env:

```
ELEVENLABS_API_KEY=xxxx
```

10. TESTES LOCAIS

10.1 Testar frontend

```
cd frontend  
npm run build
```

Se build falhar, corrigir erros de TypeScript.

10.2 Testar backend

```
cd backend  
python -m pytest # se houver testes  
curl http://localhost:8000/health
```

10.3 Testar fluxo completo

1. Abrir <http://localhost:3000/captura>
2. Preencher nome + e-mail
3. Clicar em “Baixar Grátis”
4. Verificar se lead foi criado:

```
curl http://localhost:8000/api/leads
```

5. Verificar se redirecionou para /obrigado

10.4 Testar API de pagamento (Stripe)

```
# Usar Stripe CLI para testar webhook
stripe listen --forward-to localhost:8000/api/webhook/pagamento
```

```
# Em outro terminal, simular evento
stripe trigger checkout.session.completed
```

11. DEPLOY EM PRODUÇÃO

11.1 Opção A: Docker (VPS)

```
# Na VPS
git clone https://github.com/Heverton-web/proj_fabrica-de-livros.git
cd proj_fabrica-de-livros/vendas
python scripts/criar-maquina-vendas.py <slug>

cd marketing/maquinas/<slug>
```

```
# Configurar .env com credenciais reais
cp .env.example .env
nano .env # editar
```

```
# Subir containers
docker-compose up -d
```

```
# Verificar
docker-compose ps
curl http://localhost/api/health
```

Serviços que sobem: | Serviço | Porta | Função | |---| | frontend | 3000 | Next.js | | backend | 8000 | FastAPI |
| worker-emails | --- | Processador de e-mails | | worker-leads | --- | Lead hunter | | monitor | --- | Funnel monitor
| | nginx | 80/443 | Proxy reverso |

11.2 Opção B: Vercel + Railway

Frontend → Vercel

```
cd frontend
vercel login
vercel --prod
```

Configurar env vars no dashboard Vercel: - NEXT_PUBLIC_API_URL = URL do backend Railway -
NEXT_PUBLIC_STRIPE_KEY = pk_live_...

Backend → Railway

```
cd backend
railway login
railway init
railway up
```

Configurar env vars no dashboard Railway: - DATABASE_URL = sqlite:///database/maquina.db - SMTP_* =
credenciais - STRIPE_* = credenciais

11.3 Opção C: VPS com Nginx + PM2

```
# Instalar PM2
npm install -g pm2

# Na VPS
cd marketing/maquinas/<slug>

# Backend
cd backend
pip install -r requirements.txt
pm2 start "uvicorn app.main:app --host 0.0.0.0 --port 8000" --name "mv-api"

# Frontend
cd ../frontend
npm install
npm run build
pm2 start npm --name "mv-web" -- start

# Workers
cd ..
pm2 start scripts/email_sender.py --name "mv-emails" --interpreter python
pm2 start scripts/lead_hunter.py --name "mv-leads" --interpreter python
pm2 start scripts/funnel_monitor.py --name "mv-monitor" --interpreter python

# Salvar config PM2
pm2 save
pm2 startup

# Nginx
sudo nano /etc/nginx/sites-available/mv-<slug>
```

Configuração Nginx:

```
server {
    listen 80;
    server_name seudominio.com;

    location / {
        proxy_pass http://localhost:3000;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
        proxy_set_header Host $host;
        proxy_cache_bypass $http_upgrade;
    }

    location /api/ {
        proxy_pass http://localhost:8000;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
    }
}

sudo ln -s /etc/nginx/sites-available/mv-<slug> /etc/nginx/sites-enabled/
sudo nginx -t
```

```
sudo systemctl reload nginx
```

```
# SSL com Let's Encrypt
```

```
sudo apt install certbot python3-certbot-nginx
```

```
sudo certbot --nginx -d seudominio.com
```

11.4 Script de deploy automático

```
bash scripts/deploy.sh
```

O script: 1. Faz backup do banco 2. Faz git pull 3. Build do frontend 4. Instala dependências do backend 5. Reinicia serviços via PM2 6. Verifica health check 7. Rollback se falhar

12. PÓS-DEPLOY: OPERAÇÃO 24/7

12.1 Ativar automações

```
# No diretório da máquina
```

```
python scripts/lead_hunter.py --iniciar
```

```
python scripts/email_sender.py --iniciar
```

```
python scripts/funnel_monitor.py --iniciar
```

Ou via PM2 (já ativo se usou deploy VPS):

```
pm2 list
```

12.2 Configurar cron jobs

```
crontab -e
```

Adicionar:

```
# Lead Hunter: 3x/dia (8h, 14h, 20h)
```

```
0 8,14,20 * * * cd /path/to/maquina && python scripts/lead_hunter.py
```

```
# Email Sender: 1x/dia (9h)
```

```
0 9 * * * cd /path/to/maquina && python scripts/email_sender.py
```

```
# Funnel Monitor: a cada hora
```

```
0 * * * * cd /path/to/maquina && python scripts/funnel_monitor.py
```

```
# Backup diário: 3h da manhã
```

```
0 3 * * * cp /path/to/maquina/database/maquina.db /path/to/backups/maquina_$(date +%Y\%m\%d).db
```

12.3 Verificar logs

```
# PM2
```

```
pm2 logs mv-api
```

```
pm2 logs mv-emails
```

```
pm2 logs mv-leads
```

```
# Docker
```

```
docker-compose logs -f backend
```

```
docker-compose logs -f worker-emails
```

```
# Arquivo
```

```
tail -f logs/funnel_monitor.log
```

13. MONITORAMENTO E MANUTENÇÃO

13.1 Dashboard de métricas

Acessar: <https://seudominio.com/admin>

Métricas disponíveis: - Total de leads - Taxa de conversão por etapa - Receita total - E-mails enviados/abertos/clicques - ROAS (Return on Ad Spend)

13.2 Relatório diário

Gerado automaticamente às 7h em `marketing/maquinas/<slug>/analytics/`:

```
cat analytics/relatorio_2026-08-08.md
```

13.3 Comandos de monitoramento

```
# Status da máquina  
/status
```

```
# Métricas em tempo real  
/monitorar inteligencia-artificial-empreendedores
```

```
# Forçar auto-correção  
/corrigir inteligencia-artificial-empreendedores
```

```
# Ver logs  
pm2 logs --lines 100
```

13.4 Alertas

O `funnel_monitor.py` envia alertas quando: - Taxa de conversão cai > 20% - Leads param de chegar - E-mails não são enviados - API fica fora do ar

Alertas vão para: log + webhook (configurável).

14. ESCALANDO A MÁQUINA

14.1 Quando escalar

Sinais de que é hora: - ROAS > 2x por 7 dias consecutivos - Taxa de conversão > 5% - Mais de 100 leads/dia - Receita > R\$ 5.000/mês

14.2 Como escalar

```
# Aumentar budget de anúncios  
/escalar inteligencia-artificial-empreendedores --budget +30%
```

```
# Criar lookalike audience  
# (automático quando ROAS > 2x)
```

14.3 Escalar infraestrutura

```
# Migrar SQLite → PostgreSQL  
# 1. Exportar dados  
sqlite3 database/maquina.db .dump > dump.sql
```

```
# 2. Criar banco Postgres  
createdb maquina_vendas
```

```
# 3. Importar
psql maquina_vendas < dump.sql

# 4. Atualizar DATABASE_URL
export DATABASE_URL=postgresql://user:pass@localhost/maquina_vendas
```

14.4 Criar máquinas para novas obras

```
# Cada nova obra = nova máquina
/criar-livro Marketing Digital Avançado
/criar-maquina marketing-digital-avancado

# Agora tem 2 máquinas rodando em paralelo
```

CHECKLIST FINAL

PRÉ-REQUISITOS

- ☐ Python 3.10+ instalado
- ☐ Node.js 18+ instalado
- ☐ Git instalado
- ☐ Pandoc instalado
- ☐ Typst instalado
- ☐ LLM configurada (MiMoCode/Claude/Gemini)

CONFIGURAÇÃO

- ☐ Repositório clonado
- ☐ requirements.txt instalado
- ☐ .env configurado (SMTP, Stripe, Instagram)
- ☐ descobrir_modelos.py rodou com sucesso

criação

- ☐ Obra criada e validada
- ☐ Máquina de vendas gerada
- ☐ Banco de dados inicializado
- ☐ Configs revisados (produtos, funis, personas)

FRONTEND

- ☐ npm install rodou
- ☐ npm run dev funciona
- ☐ Página de venda abre
- ☐ Formulário de captura funciona
- ☐ Admin dashboard carrega

BACKEND

- ☐ pip install rodou
- ☐ uvicorn inicia sem erros
- ☐ /health retorna OK
- ☐ POST /api/leads funciona
- ☐ Swagger docs acessível

INTEGRAÇÕES

- ☐ Stripe webhook configurado
- ☐ Instagram token configurado
- ☐ SMTP configurado e testado

DEPLOY

- ☐ Build do frontend funciona
- ☐ Docker sobe todos os serviços
- ☐ SSL configurado (HTTPS)
- ☐ Domínio apontando

OPERAÇÃO

- ☐ Lead Hunter rodando
 - ☐ Email Sender rodando
 - ☐ Funnel Monitor rodando
 - ☐ Cron jobs configurados
 - ☐ Backups automáticos ativos
 - ☐ Dashboard acessível
 - ☐ Alertas configurados
-

Guia gerado em 2026-08-08 — Fábrica Agêntica de Publicações